Promise.A+规范

1. 第一条:Terminology(术语)
    - 1.1 `Promise是具有then行为符合本规范的对象或者函数`
    - 1.2“ thenable”是定义then方法的对象或函数。
    - 1.3 `“value”是任何合法的JavaScript值（包括undefined，ableable或promise）。`
    - 1.4“ exception”是使用该throw语句引发的值。
    - 1.5 `“reason”是表明拒绝承诺的原因的值。`
2. requireMents(要求)
    - 2.1Promise States(承诺状态)
    - > A promise must be in one of three states: pending, fulfilled, or rejected.(一个promise必须要有三个状态的其中一个Pending,fulfilled,rejected)
    - 2.1.1 When pending, a promise:(什么时候是pending)
        - `2.1.1.1 may transition to either the fulfilled or rejected state. [一个pending状态的promise可以转换为任何状态的promise,成功/失败]`
    - 2.1.2 When fulfilled, a promise: (什么时候成功)
        - `2.1.2.1 must not transition to any other state.(如果成功,就不能转换成其他状态)`
        - `2.1.2.2 must have a value, which must not change.(有一个value值,永远不能改变)`
    - 2.1.3 When rejected, a promise: (什么时候失败)
        - ``2.1.3.1 must not transition to any other state. (如果失败,就不能转换成其他状态)
        - 2.1.3.2 must have a reason, which must not change. (有一个reason值,永远不能改变)``
        - > Here, “must not change” means immutable identity (i.e. ===), but does not imply deep immutability.(这里的’不能改变’是不能改变身份,但是不意味着深层的不变性
    - 2.2 Promise Methods
        - A promise must provide a then method to access its current or eventual value or reason.(一个promise必须提供一个then方法,访问其当前或者最终

value/reason的方法)
    - A promise's then method accepts two arguments:(promise方法接受2个参数)
    - ```
      promise.then(onFulfilled, onRejected)
      ```
    - 2.2.1 Both `onFulfilled` and `onRejected` are optional arguments:(成功函数和失败函数都是可选参数)
    - 2.2.1.1. If `onFulfilled` is not a function, it must be ignored.(如果成功函数不是一个函数,必须忽略他)
    - 2.2.1.2 If `onRejected` is not a function, it must be ignored.(如果失败函数不是一个函数,必须忽略他)
  - 2.2.2. If onFulfilled is a function:(如果成功函数是一个函数)
    - 2.2.2.1 it must be called after promise is fulfilled, with promise's value as its first argument.(它必须在promise完成后被调用，promise的值作为它的第一个参数)
    - 2.2.2.2 it must not be called before promise is fulfilled.(在promise实现之前不能调用它)
    - 2.2.2.3 it must not be called more than once.(他只能被调用一次)
  - 2.2.3 If onRejected is a function,(如果onRejected是个函数)
    - 2.2.3.1 it must be called after promise is rejected, with promise's reason as its first argument.(它必须在promise被拒绝后被调用，以承诺的reason作为它的第一个论据。)
    - 2.2.3.2 it must not be called before promise is rejected.(在promis被拒绝之前,他不能被调用)
    - 2.2.3.3 it must not be called more than once.(他只能被调用一次)
  - 2.2.4 `onFulfilled` or `onRejected` must not be called until the execution context stack contains only platform code. [3.1].(onFulfilled或onRejected在执行上下文堆栈仅包含平台代码之前不得调用)
  - 2.2.5 onFulfilled and onRejected must be called as functions (i.e. with no this value). [3.2][成功或者失败函数必须是函数调用,即没有this值]
  - 2.2.6 then may be called multiple times on the same promise.(then在一个promise中可以被调用多次)
    - 2.2.6.1 If/when promise is fulfilled, all respective onFulfilled callbacks must execute in the order of their originating calls to then.(如果/何时promise完成，则所有各自的onFulfilled回调必须按其对的原始调用的顺序执行then)
    - 2.2.6.2 If/when promise is rejected, all respective onRejected callbacks must execute in the order of their originating calls to then.(如果/何时promise被拒绝，则所有相应的onRejected回调必须按照其对的原始调用的顺序执行then)
  - 2.2.7 then must return a promise [3.3].(then必须返回一个promise)
    - ```
      promise2 = promise1.then(onFulfilled, onRejected);
      ```

- 2.2.7.1 If either onFulfilled or onRejected returns a value x, run the Promise Resolution Procedure [Resolve].(如果有一个onFulfilled或onRejected返回一个值x，请运行Promise Resolution Procedure [Resolve],就是一个解析函数,解析x的值)
- 2.2.7.2 If either onFulfilled or onRejected throws an exception e, promise2 must be rejected with e as the reason.(如果任何一个onFulfilled或onRejected引发异常e，则promise2必须以其e为理由予以拒绝,其实就是返回原有promise1的reason值到promise1的reject)
- 2.2.7.3 If onFulfilled is not a function and promise1 is fulfilled, promise2 must be fulfilled with the same value as promise1.(如果onFulfilled不是函数且promise1已实现，则promise2必须使用与相同的值来实现promise1。)
- 2.2.7.4 If onRejected is not a function and promise1 is rejected, promise2 must be rejected with the same reason as promise1.(如果onRejected不是功能而promise1被拒绝，则promise2必须以与相同的理由将其拒绝promise1。)

- 2.3 The Promise Resolution Procedure(承诺解决程序)
  - > The promise resolution procedure is an abstract operation taking as input a promise and a value, which we denote as [Resolve]. If x is a thenable, it attempts to make promise adopt the state of x, under the assumption that x behaves at least somewhat like a promise. Otherwise, it fulfills promise with the value x.(翻译不准确,大概理解就是x是一个成功值就返回,不是就继续调用自己?
  - > This treatment of thenables allows promise implementations to interoperate, as long as they expose a Promises/A+-compliant then method. It also allows Promises/A+ implementations to “assimilate” nonconformant implementations with reasonable then methods.(只要是符合规范的then就可以互相操作)
  - 2.3.1 If promise and x refer to the same object, reject promise with a TypeError as the reason.(如果promise和x指向同一对象，promise则以拒绝TypeError为理由。)
  - 2.3.2 If x is a promise, adopt its state [3.4]:(如果x是一个promise，则采用其状态)
    - 2.3.2.1 If x is pending, promise must remain pending until x is fulfilled or rejected.(如果x未决，则promise必须保持未决状态，直到x实现或被拒绝)
    - 2.3.2.2 If/when x is fulfilled, fulfill promise with the same value.(如果/何时x满足，promise则以相同的value返回。)
    - 2.3.2.3 If/when x is rejected, reject promise with the same reason.(如果/何时x被拒绝，promise则以相同的理由拒绝reason。)

- 2.3.3 Otherwise, if x is an object or function,(如果x是对象或函数，)
  - 2.3.3.1 Let then be x.then. 3.5
  - 2.3.3.2 If retrieving the property x.then results in a thrown exception e, reject promise with e as the reason.(如果检索属性x.then中抛出的异常的结果e，拒绝promise与e作为的原因)
  - 2.3.3.3 If then is a function, call it with x as this, first argument resolvePromise, and second argument rejectPromise, where:(如果then是函数，请使用xas this，第一个参数resolvePromise和第二个参数进行调用rejectPromise)
    - 2.3.3.3.1 If/when resolvePromise is called with a value y, run [Resolve].(何时resolvePromise使用值调用y，请运行[Resolve])
    - 2.3.3.3.1 If/when rejectPromise is called with a reason r, reject promise with r.(如果/当rejectPromise是带一个理由r，拒绝promise与r)
    - 2.3.3.3.3 If both resolvePromise and rejectPromise are called, or multiple calls to the same argument are made, the first call takes precedence, and any further calls are ignored.(如果同时调用resolvePromise和rejectPromise，或者对同一参数进行了多次调用，则第一个调用优先，而所有其他调用均被忽略)
    - 2.3.3.3.4 If calling then throws an exception e,(如果调用then引发异常e)
      - 2.3.3.3.4.1 If resolvePromise or rejectPromise have been called, ignore it.(如果resolvePromise或rejectPromise已经被调用，则忽略它)
      - 2.3.3.3.4.2 Otherwise, reject promise with e as the reason.(否则，拒绝promise与e作为reason的原因)
- 2.3.4 If then is not a function, fulfill promise with x.(如果x不是一个对象或功能，实现promise与x)
-

  > If a promise is resolved with a thenable that participates in a circular thenable chain, such that the recursive nature of [Resolve] eventually causes [Resolve] to be called again, following the above algorithm will lead to infinite recursion. Implementations are encouraged, but not required, to detect such recursion and reject promise with an informative TypeError as the reason(如果使用参与循环的可循环链的可循环变量解决了一个诺言，使得[Resolve]最终的递归性质最终导致[Resolve]再次被调用，那么遵循上述算法将导致无限递归。鼓励但不是必需的实现，以检测这种递归并promise以提供信息TypeError的理由拒绝。) 感觉就是死循环就报个类型错误

3. Notes(笔记)
4. 实用性检测

> promises-aplus-tests 插件
> npm install promises-aplus-tests -g

```
Promise源码里需要填写这几行代码,才可使用插件
Promise.defer = Promise.deferred  = function(){
    let dfd = {}
    dfd.promise = new Promise((resolve,reject)=>{
        dfd.resolve = resolve;
        dfd.reject = reject;
    })
    return dfd
}
```

> 在当前路径下 promises-aplus-tests promise.js